Part 1: Onboarding

- Basic Code Screening — 5 sections • 63 min
- Mission Rear - Introductory Course — 11 sections • 18 min
- Mission Rear - Screening — 4 sections • 18 min
- Mission Rear - Prompt Writing — 11 sections • 18 min
- Mission Rear - Attempter Intro — 10 sections • 1 min
- Mission Rear - Common Error — 4 sections • 14 min

Part 2: Tasking

Introduction

Goal of this project

- The goal of this project is to rigorously evaluate a model's ability to transition code from "State 1" (buggy or underperforming) to "State 2" (correct and ideal). By creating verification scripts and rubrics, we aim to ensure that code changes are functionally correct, do not break existing features, and adhere to design-agnostic standards.

What this module will cover

- This course will cover the three core task types (Repo Evolution Replay, Multi-Component Features, and Full Tool Implementation) and the detailed verification workflows required for each. You will learn how to set up the Docker environment, write "Pass-to-Pass" (Regression) and "Fail-to-Pass" (Functional) evaluation scripts, and navigate the "Golden Patch" to verify solution correctness.

Project Overview and Task Steps

Project Overview

The core mission of this project is to evaluate the quality of code changes by measuring the transition between two distinct states. We shouldn't just check if the final code works; we should verify the journey and the integrity of that change. While we tackle different PR Categories, the overall process remains the same. We verify this transition using 3 Verification Layers:

- **Regression:** Did we break anything in State 1?
- **Functional**: Did we actually reach State 2?
- **Semantic:** Is the code high-quality "under the hood"?

Task Steps

Every task begins with a standard set of inputs. You will receive:

- **Task Type:** One of the three specs (Repo Evolution, Multi-Component, or Full Tool).
- **Repo Assets:** Git Repository URL and a Dockerfile to set up your environment.
- **The "Golden" Assets:** A Golden Patch URL (the ideal solution), a PR Message Link, and links to Commit 1 (start) and Commit 2 (end).

- **The "Golden" Assets:** A Golden Patch URL (the ideal solution), a PR Message Link, and links to Commit 1 (start) and Commit 2 (end).

Once you have your assets, you will follow this execution path:

- **Step 1: Repository Setup:** Review the provided GitHub Repo and Docker configuration.
- **Step 2: Complexity Validation:** Validate that the issue meets the project's complexity standards (e.g., Logic Depth, Scope).
- **Step 3: Prompt Engineering:** Write a detailed user prompt that forces the specific architectural choices seen in the Golden Patch.
- **Step 4: Test Verification & Scripting:**
  - Create **Run Script (`run_script.sh`)** : Executes the tests.
  - Create **Parsing Script (`parsing.py`)** : Converts test logs (stdout/stderr) into JSON.
  - **Golden Loop:** Run these scripts against the Golden Patch. If it fails, you must **rewrite the Golden Patch** or modify tests until it passes.
  - **Submit:** Upload the **`run_script.sh`**, **`parsing.py`**, and the generated **`output.json`**.
- **Step 5: Rubric Generation:** Create a comprehensive scoring guide (rated 1-5) to assess Functional Correctness, Test Verification, and Code Quality.
- **Step 6: Final Complexity:** Categorize the overall difficulty (Medium, High, or Expert).

Task Types

Task Types

The workflow verification strategy changes depending on the **Task Type** you are assigned. We categorize tasks into three types, each requiring a different verification focus and environment setup. While the core process of validating the **Golden Patch** remains the same, the nature of the tests you write will differ significantly between fixing a bug in an existing repo and building a new tool from scratch. These are the three possible **Task Types:**

1. **Repo Evolution:**
    - **The Goal**: Evaluate a model's ability to implement features between two versions (v1 → v2) without breaking existing functionality.
    - **Environment:** You **must** pre-load the "v1" codebase and seed the database.
    - **Verification Strategy: Pass-to-Pass (P2P) + Pass-to-Pass (F2P).**
        - **Regression (P2P):** This is your **critical priority**. You must ensure the existing v1 test suite passes 100% on v2 to prove no regressions occurred.
        - **Migration (F2P):** You must also provide Fail-to-Pass tests that verify the *change* itself (e.g., tests that fail on v1 because the new library isn't there, but pass on v2).
2. **Multi-Component Features:**

2. **Multi-Component Features:**
    - **The Goal**: Verify seamless integration and correct data persistence across multiple layers (e.g., Frontend → API → Database).
    - **Environment:** Like Repo Evolution, you **must** pre-load the "v1" codebase and seed the database.
    - **Verification Strategy: P2P + F2P with an End-to-End Focus (E2E).**
        - **Integrity (F2P):** You must run full transaction flows (End-to-End) and query the database directly to confirm data integrity (e.g., verifying an order row was actually created).
        - **Stability (P2P):** Ensure that touching multiple files didn't break adjacent features.
3. **Full Tool Implementation:**
    - **The Goal**: Build a design-agnostic tool from scratch that meets specific functional requirements.
    - **Environment:** You should initialize an empty directory or a scaffold skeleton.
    - **Verification Strategy: F2P + Rubrics.**
        - **Functional (F2P):** Since this is a new tool, your scripts will primarily be Fail-to-Pass (verifying the new features work).
        - **Rubrics:** You must use Abstract Syntax Tree (AST) analysis to ensure the tool is design-agnostic (e.g., checking for hardcoded dependencies or specific class structures).

Environment & Setup Strategy

Environment & Setup Strategy

Depending on which of the three task types your task falls into, you must determine the correct starting state for your environment. For **Repo Evolution** and **Multi-Component Features** tasks, you must adopt a "State 1" setup. You are required to pre-load the "v1" codebase, which represents the state of the repository before any changes were made. Furthermore, if the application uses a database, you must seed the database with valid test data. You cannot verify a migration or a complex feature integration if the system is empty or lacks the necessary historical context.

In contrast, **Full Tool Implementation** tasks require a "Greenfield" approach. You should initialize an empty directory or a basic scaffold skeleton. Do not inherit legacy data or configurations unless explicitly told to do so. Your focus here is on a clean and design-agnostic implementation.

Common Issue that leads to task failure: Be aware to avoid the common error of running a "Repo Evolution" task on an empty folder or a "Full Tool" task on a cluttered legacy codebase. Always check your Task Type first to determine if you need the historical data of v1 or a clean slate.

Testing Protocols: P2P vs. F2P

You must create a `run_script.sh` that executes specific test suites to verify the transition from State 1 to State 2. These tests fall into two distinct protocols: Pass-to-Pass (P2P) and Fail-to-Pass (F2P).

- **Pass-to-Pass (P2P): The Regression Check**: This protocol ensures that you have not broken existing functionality. You must provide a file that includes tests that passed before the patch and continue to pass after the patch. For **Repo Evolution** tasks, P2P is your primary focus. Since you are modifying an existing codebase, you must prove that the original v1 test suite still passes 100% after your changes.
- **Fail-to-Pass (F2P): The Functional Check:** This protocol verifies that the new features requested in the prompt were successfully built. You must provide a file that includes tests that *failed* before the patch (because the feature did not exist) and *pass* after the patch is applied. For **Multi-Component Features** and **Full Tool Implementation** tasks, F2P is critical. You are adding new capabilities, so you must write new tests to prove these capabilities work as expected.

In summary, if you are maintaining code (Repo Evolution), prioritize P2P to ensure stability. If you are building code (Multi-Component or Full Tool), prioritize F2P to prove implementation.

The Golden Patch Workflow

The Golden Patch Workflow

You might assume the "Golden Patch" provided to you is perfect. In this project, that is a dangerous assumption. You are responsible for proving it is perfect by running it through the "Golden Loop."

Instead of a linear path, you must follow this iterative cycle:

1. **Write Tests:** Create your P2P and F2P scripts based strictly on the Prompt you wrote.
2. **Run Tests:** Execute your scripts against the provided **Golden Patch**.
3. **The Decision Point:**
    1. **If it Fails:** You must **rewrite the Golden Patch**. Do not lower your test standards to make it pass. If the provided solution does not meet your prompt's requirements, you are authorized and required to fix the code until it does.
    2. **If it Passes:** Only then are you ready to generate the final `output.json` and submit your files.

Rubric Generation & Quality Standards

Once your Golden Patch is verified, you must create a comprehensive scoring guide (Step 5 in the UI) to assess whether a model's solution fulfills the prompt. You are not just checking "Pass/Fail"; you are defining quality levels on a 1–5 scale.

The 1–5 Rating Scale: You must define criteria for every level of this scale. Use the following standard definitions as your baseline:

- **1 - Failure**: The code fails to run, breaks the build, or hallucinates non-existent libraries.
- **2 - Poor**: Addresses part of the prompt but misses major constraints (e.g., uses the wrong library version).
- **3 - Acceptable**: Functionally correct but lacks optimization or misses edge cases found in the Golden Patch.
- **4 - Good**: Meets all functional requirements and passes tests; code is clean and idiomatic.
- **5 - Excellent**: Perfect implementation. Matches the Golden Patch in logic, style, and efficiency.

Anatomy of a Good Rubric: A rubric is only useful if it is objective and clear. When writing your criteria, ensure they meet these 7 "Good Rubric" characteristics:

1. **Prompt-Grounded**: Your criteria must come *only* from explicit requirements in the prompt. Do not grade based on hidden preferences.
2. **Verifiable**: The rule must be checked directly against the response. If you can't prove it, don't include it.
3. **Atomic**: Test one single requirement per rubric line. Do not bundle multiple checks into one complex rule.
4. **Self-Contained**: The rubric must be understandable on its own without requiring extra context.
5. **Non-Overlapping**: Do not write two rubrics that check the same thing.

Anatomy of a Good Rubric: A rubric is only useful if it is objective and clear. When writing your criteria, ensure they meet these 7 "Good Rubric" characteristics:

1. **Prompt-Grounded**: Your criteria must come *only* from explicit requirements in the prompt. Do not grade based on hidden preferences.
2. **Verifiable**: The rule must be checked directly against the response. If you can't prove it, don't include it.
3. **Atomic**: Test one single requirement per rubric line. Do not bundle multiple checks into one complex rule.
4. **Self-Contained**: The rubric must be understandable on its own without requiring extra context.
5. **Non-Overlapping**: Do not write two rubrics that check the same thing.
6. **Specific**: Avoid vague language (like "good code"). Be concrete and auditable so that any reviewer would give the same score.
7. **Clear & Positive**: State what the response *should* include using simple, direct language.

Required Categories: You should organize your rubrics into three specific categories:

- **Functional Correctness**: Does it solve the issue described in the PR?
- **Test Verification**: Do the unit tests pass?
- **Code Quality**: Is the code efficient and readable?

Submission & Final Complexity

Once your Golden Patch has passed the "Golden Loop" and your Rubrics are defined, you are ready for the final stage. This involves uploading your artifacts and assigning a final complexity rating to the task.

The Submission Package: You must upload three specific files to complete the technical portion of the task:

- **Run Script (** `run_script.sh` **)** : The shell script that sets up the environment and executes your tests inside the Docker container.
- **Parsing Script (** `parsing.py` **)** : The Python script that strictly converts the stdout/stderr logs from your test runner into a standardized JSON format.
- **Output JSON (** `output.json` **)** : The actual result file generated by running the Golden Patch through your scripts.

Complexity Categorization: The final step requires you to categorize the overall difficulty of the task into one of three levels. This rating helps the project track the distribution of work complexity.

- **Medium (Moderate Difficulty)** : Select this for tasks with limited scope, such as standard library updates or logic changes contained within a single file.
- **High (Challenging)** : Select this for tasks requiring significant expertise, such as major version migrations or architectural changes that touch multiple files.
- **Expert (Very Challenging)** : Select this only for tasks requiring deep expertise, such as kernel-level changes, complex security implementations, or proprietary algorithms.

The Justification: You cannot simply pick a level; you must defend it. In the "Complexity Justification" field, explain your choice based on specific factors: the number of lines of code changed, the conceptual difficulty of the logic, and the complexity of the dependencies involved.

Quiz

You have been assigned a "Full Tool Implementation" task. What is the correct strategy for initializing your environment?

Attempts remaining: 0

- Pre-load the v1 codebase and seed the database.
- Initialize an empty directory or scaffold. ✓
- Copy the Golden Patch immediately into the root folder.
- Download the production database dump.

This is correct

Which testing protocol is primarily used for "Repo Evolution" tasks to ensure that the transition didn't break existing functionality?

Attempts remaining: 0

- Fail-to-Pass (F2P).
- Pass-to-Pass (P2P). ✓

- Design-Agnostic Rubrics.
- Complexity Validation.

This is correct

You run your evaluation scripts against the provided Golden Patch, and the tests fail. What is the mandatory next step?

Attempts remaining: 0

- Submit the files and note the failure in the JSON output.
- Lower the strictness of your tests until they pass.
- Rewrite the Golden Patch until it passes your tests. ✓
- Skip the verification loop and move to Rubric Generation.

This is correct

What is the specific purpose of the `parsing.py` file in the final submission package?

Attempts remaining: 0

- To execute the unit tests inside the Docker container.
- To convert test logs (stdout/stderr) into a standardized JSON format. ✓
- To perform AST analysis and check for code style violations.
- To generate the complexity justification for the task.

This is correct

In the Rubric Generation step, what does it mean for a rubric criterion to be "Atomic"?

Attempts remaining: 0

- It allows for subjective interpretation by the reviewer.
- It checks for security vulnerabilities specifically.
- It is grounded in hidden preferences not found in the prompt.
- It tests one single requirement per line, not multiple. ✓

This is correct

Conclusion

Thank you for completing Mission Rear - Introductory Course.

Recap

In this module, we established the core framework for evaluating code transitions from **"State 1"** (buggy or incomplete) to **"State 2"** (correct and feature-complete). We covered the critical distinctions between the three task types:

- **Repo Evolution** and **Multi-Component Features**, which require pre-loading the v1 codebase and running **Pass-to-Pass (P2P)** regression tests.
- **Full Tool Implementation**, which requires a greenfield setup and relies heavily on **Fail-to-Pass (F2P)** functional tests and **Rubrics**.

Finally, we detailed the **Golden Patch Workflow**, emphasizing that you must test the provided solution against your prompt and **rewrite the Golden Patch** if it fails your verification standards.